

Figure 2: The FLAG-TRADER pipeline for financial trading, utilizing an LLM-based actor-critic architecture. The LLM consists of **frozen base layers** θ_{frozen} that retain pre-trained knowledge and **trainable top layers** θ_{train} for financial decision-making. Both the POLICY_NET and VALUE_NET share these trainable layers while maintaining separate *policy head* θ_P and *value head* θ_V , which are updated by policy gradient method.

4 FLAG-TRADER

To tackle the challenge of directly fine-tuning an LLM for both alignment and decision-making, we introduce FLAG-TRADER, a fused LLM-agent and RL framework for financial stock trading. In FLAG-TRADER, a partially fine-tuned LLM serves as the policy network, leveraging its pre-trained knowledge while adapting to the financial domain through parameter-efficient fine-tuning, as shown in Figure 2. The model processes financial information using a textual state representation, allowing it to interpret and respond to market conditions effectively. Instead of fine-tuning the entire network, only a subset of the LLM’s parameters is trained, striking a balance between adaptation and knowledge retention. In the following, we will present the prompt input design and the detailed architecture of FLAG-TRADER.

4.1 Prompt Input Design

The first stage of the pipeline involves designing a robust and informative prompt, denoted as $\text{lang}(s_t)$, which is constructed based on the current state s_t to guide the LLM in making effective trading decisions. The prompt is carefully structured to encapsulate essential elements that provide context and ensure coherent, actionable outputs. It consists of four key components: a *task description*, which defines the financial trading objective, outlining the problem domain and expected actions; a *legible action space*, specifying the available trading decisions (Sell, Hold, Buy); a *current state representation*, incorporating market indicators, historical price data, and portfolio status to contextualize the decision-making process; and an *output action*, which generates an executable trading decision. This structured prompt ensures that the LLM receives comprehensive input, enabling

it to produce well-informed and actionable trading strategies, as illustrated in Figure 3.

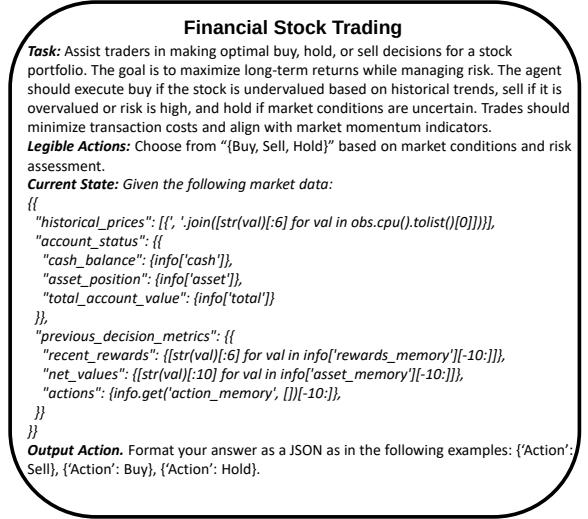


Figure 3: The format of input prompt. It contains the task description, the legible action set, the current state description, and the output action format.

4.2 FLAG-TRADER Architecture

To incorporate parameter-efficient fine-tuning into the policy gradient framework, we partition the intrinsic parameters of the LLM into two distinct components: the frozen parameters inherited from pretraining, denoted as θ_{frozen} , and the trainable parameters, denoted as θ_{train} . This separation allows the model to retain general language understanding while adapting to financial decision-making with minimal computational overhead. Building upon this LLM structure, we introduce a policy network and a value network, both of which leverage the trainable top layers of the LLM for domain adaptation while sharing the frozen layers for knowledge retention. The overall architecture is illustrated in Figure 2.

4.2.1 Policy Network Design

The policy network is responsible for generating an optimal action distribution over the trading decision space $\mathcal{A}$, conditioned on the observed market state. It consists of three main components:

State Encoding. To effectively process financial data using the LLM, the numerical market state s is first converted into structured text using a predefined template¹

$$\text{lang}(s) = \text{"Price: } \$p, \text{ Vol: } v, \text{ RSI: } r, \dots\text{"}. \quad (4)$$

This transformation enables the model to leverage the LLM’s textual reasoning capabilities, allowing it to understand and infer trading decisions in a structured, language-based manner.

LLM Processing. The tokenized text representation of the state is then passed through the LLM backbone, which consists of: 1) **Frozen layers** (preserve general knowledge): Token embeddings $E = \text{Embed}(\text{lang}(s))$ pass through LLM frozen layers, i.e.,

$$h^{(1)} = \text{LLM}_{1:N}(E; \theta_{\text{frozen}}). \quad (5)$$

These layers preserve general knowledge acquired from pretraining, ensuring that the model maintains a strong foundational understanding of language and reasoning. 2) **Trainable layers** (domain-specific adaptation): The output from the frozen layers is then passed through the trainable layers, which are fine-tuned specifically for financial decision-making, i.e.,

$$h^{(2)} = \text{LLM}_{N+1:N+M}(h^{(1)}; \theta_{\text{train}}). \quad (6)$$

This structure enables efficient adaptation to the financial domain without modifying the entire LLM, significantly reducing training cost while maintaining performance.

Policy Head. Finally, the processed representation is fed into the policy head, which outputs a probability distribution over the available trading actions according to

$$\text{logits} = \text{POLICY_NET}(h^{(2)}, \theta_P) \in \mathbb{R}^{|\mathcal{A}|}, \quad (7)$$

where θ_P is the parameter of POLICY_NET, with action masking for invalid trades:

$$\pi(a|s) = \begin{cases} 0 & a \notin \mathcal{A}, \\ \frac{\exp(\text{logits}(a))}{\sum_{a' \in \mathcal{A}} \exp(\text{logits}(a'))} & \text{otherwise.} \end{cases} \quad (8)$$

¹To simplify notation, we use $\text{lang}(s_t)$ to represent both the state encoding and the prompt, acknowledging this slight abuse of notation for convenience.

This ensures that actions outside the valid set $\mathcal{A}$ (e.g., selling when no stocks are held) have zero probability, preventing invalid execution.

4.2.2 Value Network Design

The value network serves as the critic in the RL framework, estimating the expected return of a given state to guide the policy network’s optimization. To efficiently leverage the shared LLM representation, the value network shares the same backbone as the policy network, processing the textual state representation through the frozen and trainable layers (4)–(6). This design ensures efficient parameter utilization while maintaining a structured and informative state encoding. After passing through the LLM processing layers, the output $h^{(2)}$ is fed into a separate value prediction head, which maps the extracted features to a scalar value estimation:

$$V(s) = \text{VALUE_NET}(h^{(2)}, \theta_V) \in \mathbb{R}^1, \quad (9)$$

where θ_V is the parameter of VALUE_NET.

4.3 Online Policy Gradient Learning

The policy and value networks in FLAG-TRADER are trained using an online policy gradient approach, ensuring that the model continuously refines its decision-making ability. The learning process follows an iterative cycle of state observation, action generation, reward evaluation, and policy optimization. The parameters of the model are updated using stochastic gradient descent (SGD), leveraging the computed policy and value losses to drive optimization.

At each training step, we define two key loss functions, i.e., *policy loss* $\mathcal{L}_P$: measures how well the policy network aligns with the expected advantage-weighted log probability of actions; *value loss* $\mathcal{L}_V$: ensures that the value network accurately estimates the expected return.

Remark 4.1. The definitions of *policy loss* and *value loss* may vary across different actor-critic (AC) algorithms. Here, we present a general formulation for clarity and ease of expression. Notably, our framework is designed to be flexible and adaptable, making it compatible with a wide range of AC algorithms.

Based on these loss functions, the model updates the respective network parameters using backpropagation as follows.

Update Policy Head. The policy network parameters θ_P are updated via SGD to minimize the

policy loss $\mathcal{L}_P$

$$\theta_P \leftarrow \theta_P - \eta \nabla_{\theta_P} \mathcal{L}_P, \quad (10)$$

where η is the learning rate for updating policy head θ_P .

Update Value Head. The value network parameters θ_V are optimized via SGD to minimize the temporal difference (TD) error over policy loss $\mathcal{L}_V$

$$\theta_V \leftarrow \theta_V - \eta \nabla_{\theta_V} \mathcal{L}_V. \quad (11)$$

Update Trainable LLM Layers. The trainable LLM parameters θ_{train} are updated via SGD jointly based on both the policy and value losses, i.e., $\mathcal{L}_P$ and $\mathcal{L}_V$, allowing the shared LLM representation to align with optimal decision-making:

$$\theta_{\text{train}} \leftarrow \theta_{\text{train}} - \beta \nabla_{\theta_{\text{train}}} (\mathcal{L}_P + \mathcal{L}_V), \quad (12)$$

where β is the learning rate for LLM parameter θ_{train} .

The updates in (10)–(12) are performed iteratively until the stopping criteria are met, as outlined in Algorithm 1. This iterative learning process effectively balances exploration and exploitation, enhancing policy performance while maintaining stability. To mitigate overfitting and policy divergence, we employ Proximal Policy Optimization (PPO), which constrains updates by limiting the divergence from previous policies, ensuring more controlled and reliable learning. The detailed procedure of how to compute *policy loss* $\mathcal{L}_P$ and *value loss* $\mathcal{L}_V$ can be found in Appendix A.

5 Experiments

This section describes the overall experimental design and environmental setup for comparing the performance of different trading agents under consistent conditions.

5.1 Experiment Setup

For our single-asset trading tasks, we adopt two baselines: the buy-and-hold strategy and the LLM-based trading agent from INVESTORBENCH (Li et al., 2024a), which integrates 13 proprietary or open-source large language models. Our proposed model, FLAG-TRADER (built on a 135M-parameter LLM), is then evaluated against these baselines for a comprehensive performance comparison.

We focus on five stocks and one crypto: Microsoft Corporation (MSFT), Johnson & Johnson

Algorithm 1 FLAG-TRADER

- 1: **Require:** Pre-trained LLM with parameter $\theta := (\theta_{\text{frozen}}, \theta_{\text{train}})$, environment dynamics $\mathcal{T}$, reward function $\mathcal{R}$;
 - 2: Initialize policy network θ_P and value network θ_V with shared LLM trainable layers θ_{train} ;
 - 3: Initialize experience replay buffer $B \leftarrow \emptyset$
 - 4: **for** iteration $t = 1, 2, \dots$, **do**
 - 5: Fetch the current state s_t from the environment and construct an input prompt $\text{lang}(s_t)$;
 - 6: Pass prompt $\text{lang}(s_t)$ through LLM;
 - 7: POLICY_NET outputs a_t from action space $\{\text{"buy"}, \text{"sell"}, \text{"hold"}\}$ based on (8);
 - 8: Execute action a_t in the environment and observe reward $r(s_t, a_t)$ and transition to new state s_{t+1} ;
 - 9: Store experience tuple (s_t, a_t, r_t, s_{t+1}) in replay buffer B ;
 - 10: **if** $t \bmod \tau = 0$ **then**
 - 11: Update policy head θ_P according to (10);
 - 12: Update value head θ_V according to (11);
 - 13: Update the trainable LLM layers θ_{train} according to (12).
 - 14: **end if**
 - 15: **end for**
 - 16: **Return:** Fine-tuned POLICY_NET(θ_P).
-

(JNJ), UVV Corporation (UVV), Honeywell International Inc. (HON), Tesla, Inc. (TSLA) and Bitcoin (BTC). As summarized in Table 1, each agent’s performance is measured across these assets. All language models use a temperature of 0.6 during inference to balance consistency and creativity in their responses.

We report four metrics—Composite Return (CR), Sharpe Ratio (SR), Annualized Volatility (AV), and Maximum Drawdown (MDD) and select final results from the test trajectory corresponding to the median of these metrics. If the median values arise from different epochs, we prioritize the run producing the median SR. Due to varying data availability, warm-up and test periods may differ. For the trading tasks of five stocks, the warm-up period is July 1, 2020, to September 30, 2020, and the test period is October 1, 2020, to May 6, 2021. On the other hand, the warm-up period of BTC trading is from 2023-02-11 to 2023-04-04 and the test period is from 2023-04-05 to 2023-11-05.

We deploy LLMs using the vllm framework, with configurations depending on model size.